Building a Traffic Scene in Isaac Sim

A beginner's guide to the Zaixen ring-road simulation: how it was built, how it works, and how to build and explain a scene like it yourself.



The finished scene: roads, ring road, traffic lights, NPC cars and scenery.

Written in plain English. Every topic is explained three ways: **in simple words**, **step by step**, and the **technical way**. Read whichever you need; they say the same thing at different depths.

Contents

1. The idea in one page **5**

The seven layers (build them in this order) . . . 5

2. Words you need **7**

3. Your workshop: tools and the way of working **8**

3.1 Two ways to run code in Isaac Sim . . . 8

3.2 Long jobs: fire-and-forget and result files . . . 8

3.3 The edit - run - check loop . . . 9

3.4 The biggest trap: stale Python files . . . 9

3.5 Backups and the session layer . . . 10

3.6 How the scripts run inside Isaac Sim (and why Play alone is not enough) . . . 10

3.7 How the small NPC Traffic window is built (omni.ui) . . . 11

3.8 Small everyday tricks . . . 12

4. Building the world **14**

4.1 Coordinates, units and the one height rule . . . 14

4.2 Ground and roads . . . 14

4.3 Sidewalks with rounded corners (curb returns) . . . 15

4.4 The ring road . . . 16

4.5 Road paint . . . 16

4.6 Materials and lights . . . 17

4.7 Traffic lights that change colour . . . 17

4.8 Scenery that costs almost nothing (PointInstancer) . . . 18

5. Physics: making things solid and repeatable **19**

5.1 Colliders and friction . . . 19

5.2 Three kinds of moving things . . . 19

5.3 A fixed time step: why runs repeat . . . 20

5.4 Detecting a crash (contact reports) . . . 21

6. NPC traffic: cars that behave **22**

6.1 A car's route is a closed loop of lines and arcs . . . 22

6.2 Speed: accelerate, brake, and slow for corners . . . 22

6.3 Obeying the traffic light . . . 23

6.4 Keeping a gap behind the car in front . . . 23

6.5 Crossings and merges: conflict zones . . . 23

6.6 Yielding to the ego . . . 24
6.7 The slider and the pool of cars . . . 25

7. The autonomous ego **26**

7.1 What the vehicle is made of . . . 26
7.2 Steering like a real car (Ackermann) . . . 26
7.3 The route and how the ego follows it . . . 27
7.4 The decision: one clear function . . . 28
7.5 A stand-in for a sensor . . . 29
7.6 Safety nets . . . 29

8. Cameras **31**

8.1 A camera that is locked to the ego . . . 31
8.2 Why we could not delete Camera_Chase . . . 32

9. Proving it works **33**

9.1 Test the brain without the simulator . . . 33
9.2 Soak testing: long runs with many random starts . . . 33
Bugs the tests caught (a real list) . . . 33
9.3 The in-app validator . . . 34
9.4 Checking determinism . . . 35

10. Logging your runs **36**

10.1 The CSV recorder . . . 36

11. Saving, backups and your daily routine **37**

11.1 Saving safely . . . 37
11.2 Your daily routine . . . 37

12. When something goes wrong **39**

13. How to explain this to other people **40**

The 30-second version (anyone) . . . 40
The 2-minute version (a friend who knows a little tech) . . . 40
The technical version (an engineer) . . . 40
Questions people ask and short answers . . . 40

14. Build your own from zero **42**

Ten habits that will save you days . . . 42
Exercises to try alone . . . 42

Appendix A - Cheat sheet 43

Appendix B - Key numbers 44

Appendix C - Where everything is 45

Appendix D - The maths, in one place 46

1. The idea in one page

What we built, in layers, so you always know which part you are looking at.

We built a small town in Isaac Sim (NVIDIA's robot simulator): a plus-shaped road with traffic lights, a ring road around it, lots of scenery, up to 30 computer-driven cars, and one self-driving robot car (the **ego**) that obeys the lights and avoids the cars. It runs on real physics, so if the ego hits something, it really hits it.

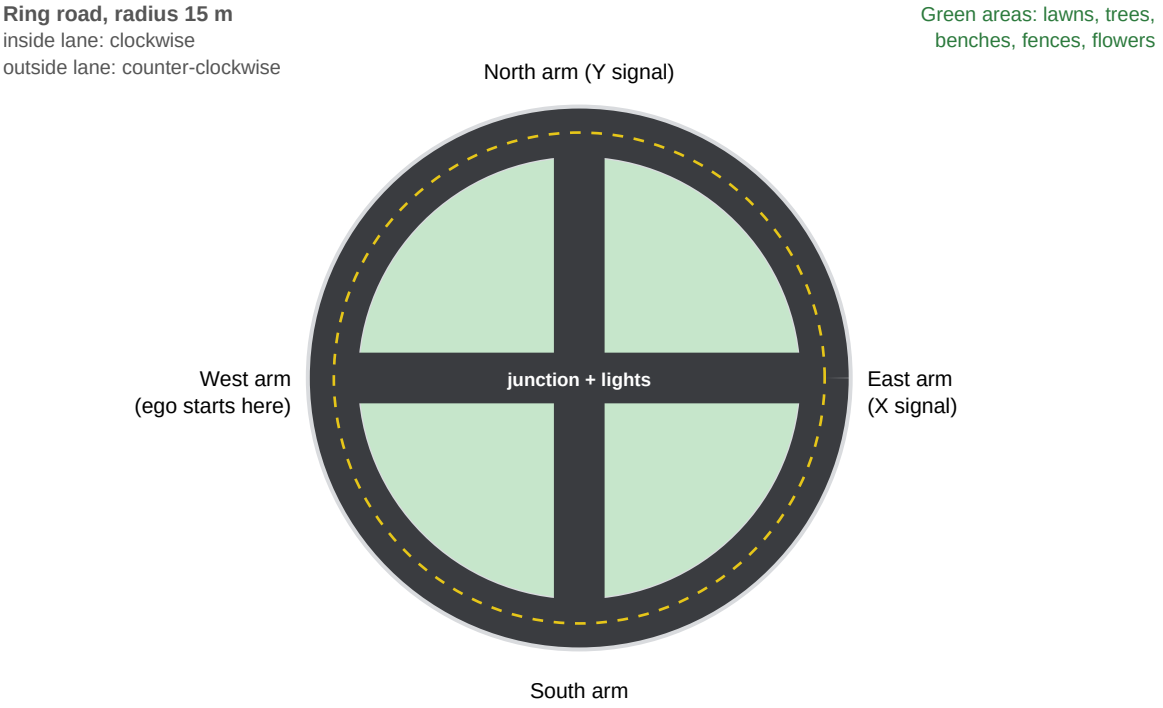


Figure 1 - Top-down sketch of the layout (not to scale).

The seven layers (build them in this order)

Layer	What it is	Why it comes in this order
1. World	Ground, roads, sidewalks, curbs, lane paint	Everything else stands on it. Get the geometry and heights right first.
2. Look	Materials, sky/sun lights, scenery (trees, benches...)	Makes it readable and pleasant. Cheap to change later.
3. Signals	Traffic lights that change colour on a timer	Cars and the ego need a signal to obey.
4. Physics	Colliders, friction, fixed time step	So things can touch, roll and repeat exactly.
5. NPC cars	Cars that follow paths, keep gaps, take turns	They make the world 'alive' and give the ego a job.
6. Ego	A vehicle that decides and drives itself	The thing you actually want to test.
7. Proof	Tests, checks, logging, saving	So you can trust it and repeat it.

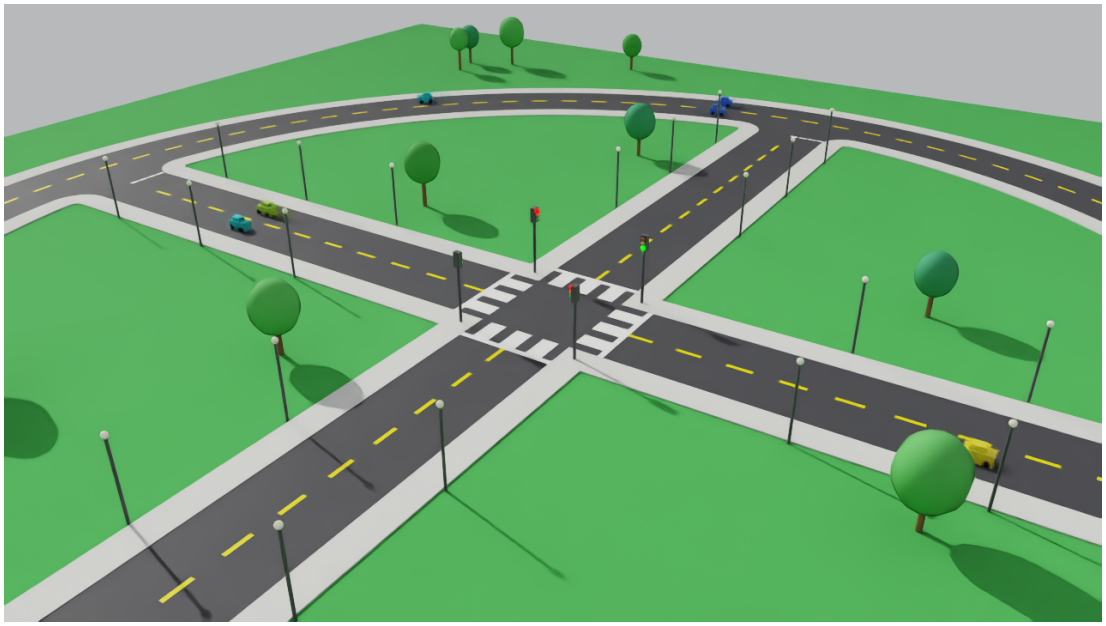


Figure 2 - A closer look: junction, lamps, lawns and cars.

Remember Almost everything you do in Isaac Sim is one of only three things: **create/edit objects** in the scene (USD), **run code every step** while the simulation plays (callbacks), or **check that it worked** (tests and screenshots). The rest of this guide is those three things, repeated.

2. Words you need

A short dictionary. Each word has a plain meaning and a precise one.

Word	In simple words	Technical meaning
USD	The file format Isaac Sim uses to describe a 3D world. Like a recipe list of every object.	Universal Scene Description (`.usd`, `.usda`, `.usdc`). A text or binary scene graph with layers, composition and a Python API (`pxr`).
Stage	The world that is currently open.	The composed scene, `omni.usd.get_context().get_stage()`. Everything you edit lives on it.
Prim	One thing in the world: a road, a car, a light, a folder.	A node of the scene graph with a path like `/World/Junction/Road_X`, a type (Cube, Mesh, Xform...) and attributes.
Attribute	A setting on a prim, like size or colour.	Typed value on a prim, e.g. `xformOp:translate`. Can be authored per time sample.
Xform / transform	Where something is, which way it faces, how big it is.	Ordered ops: translate, rotate, scale (`xformOpOrder`). Applied as scale first, then rotation, then translation.
Layer	One file of edits. The scene file is on top of other files.	USD composes many layers (root layer, sublayers, session layer). The strongest opinion wins.
Reference	'Use that other file's object here.' Like a shortcut.	A composition arc that pulls a prim hierarchy from another file. Referenced prims cannot be deleted locally, only overridden.
Session layer	A scratch page that is never saved.	An anonymous in-memory layer above the root layer. Good for safe experiments.
Material / shader	The colour and shininess of a surface.	`UsdShade.Material` with a `UsdPreviewSurface` shader (diffuseColor, roughness, emissiveColor).
Collider	The invisible 'solid' shape physics uses.	`UsdPhysics.CollisionAPI` applied to a prim so PhysX generates contacts with it.
Rigid body	A solid object physics can move.	`UsdPhysics.RigidBodyAPI`. Dynamic (pushed by forces) or kinematic (moved by you, pushes others, cannot be pushed).
Articulation, joint, drive	A robot made of parts joined by hinges; a drive is the little motor in a hinge.	A tree of rigid bodies connected by joints. `DriveAPI` gives each joint a target position and/or velocity, stiffness and damping.
Physics step	One tiny tick of the physics clock.	The simulation advances by a fixed `dt`. We use 1/60 s. Callbacks can run on every step.
Determinism	Same start, same result, every time.	No wall-clock time, fixed step, seeded random numbers, and a reset to an identical start state.
PointInstancer	One tree model stamped 500 times cheaply.	`UsdGeom.PointInstancer`: prototypes plus arrays of positions, orientations, scales and prototype indices.
Callback / subscription	A function Isaac calls for you again and again.	Registered with the app or physics (e.g. `subscribe_physics_step_events`). Must be unsubscribed and kept referenced.
Ackermann steering	How real cars steer: the inside wheel turns more than the outside wheel.	Geometry where all wheel axes meet at one point, so no tyre scrubs sideways in a turn.
Pure pursuit	'Steer toward a point a little way ahead on the route.'	A path-tracking rule: steering angle = atan(2 L sin(alpha) / Ld).
Soak test	Run the simulation for hours with different random starts to catch rare bugs.	Long randomised runs with automatic pass/fail metrics.

3. Your workshop: tools and the way of working

How to talk to Isaac Sim, run code in it, look at the result, and not lose your work.

3.1 Two ways to run code in Isaac Sim

IN SIMPLE WORDS

You can type or paste code inside Isaac Sim (the Script Editor), or you can send code to it from outside using a small helper. The helper is the better habit because your scripts stay as normal files you can edit with any editor and keep in the project folder.

STEP BY STEP

1. Inside the app: open `Window > Script Editor`, open your `.py` file, press Run (Ctrl+Enter).
2. From outside: make sure Isaac Sim was started with its Python server on. It listens on `127.0.0.1` port `8226`.
3. Send a file: `python isaacsim_send.py --file your_script.py` (the helper lives in `skills/isaac-sim-remote/scripts`).
4. Send one line: `python isaacsim_send.py "print(1+1)"`.
5. For anything that takes longer than about a minute add `--fire-and-forget` (see next topic).

TECHNICAL WAY

The server executes your code inside the running Kit process, on the main thread, with the stage open. Your script can `import omni.usd`, `pxr` and so on.

- `--file` runs the file in an isolated function scope; use a named `--context` if variables must persist between calls.
- Top-level `await` is allowed, so you can `await omni.kit.app.get_app().next_update_async()` to let frames pass.
- Values are passed in with `--arg name=value` (they arrive as Python globals).

3.2 Long jobs: fire-and-forget and result files

IN SIMPLE WORDS

If a job takes minutes, the connection to Isaac Sim can drop and you lose the printout. So you start the job, let it run inside Isaac Sim, and make it write its results to a text file. You read the file later.

STEP BY STEP

1. Write your script so it opens an output file (for example in your project `logs` folder) and writes lines as it goes.
2. Send it with `--fire-and-forget`. You get a task id straight away.
3. Poll: keep checking until the file contains the word `DONE`.
4. If the file never appears, ask the server: `python isaacsim_send.py --introspect tasks`. It lists each task as `ok` or `error`.
5. If a task shows `error`, run the same script once in the normal (foreground) mode. The error message will print.

TECHNICAL WAY

The client socket times out after roughly 60 to 90 seconds even when the server is still working.
Fire-and-forget returns an ACK with a task id and the code continues in the background.
Errors in background tasks are not shown anywhere except `--introspect tasks` (status only), so always test a new script in the foreground first, with a short run.

3.3 The edit - run - check loop

IN SIMPLE WORDS

Work in tiny steps: change one thing, run it, and look. Never build ten things and hope they all work.

STEP BY STEP

1. Edit a script in your editor.
2. Send it to Isaac Sim.
3. Check with numbers first (print prim positions, counts, attribute values).
4. Then check with your eyes: take a screenshot (top view and a close-up).
5. If it is right, move on. If not, fix that one thing.
6. After a good step, save a backup copy of the scene file.

TECHNICAL WAY

Numbers are faster and more reliable than eyes: e.g. read back `top of every road == 0.02`.
Screenshots catch what numbers miss (a wrong rotation, a floating object).
Screenshots in this project use `isaacsim.test.utils.image_capture.capture_viewport_screenshot_async` after setting the viewport camera with `ViewportManager.set_camera_view(eye, target)`. A top-down camera has no fixed 'up', so the picture may be rolled 90 degrees (in our top views, image-horizontal is world Y).

3.4 The biggest trap: stale Python files

IN SIMPLE WORDS

Isaac Sim can silently keep using an OLD copy of your script after you edit it. You will change the code, run it, see no change, and wonder why. The fix is to make sure Isaac loads your file fresh every time.

STEP BY STEP

1. Keep your own modules (`zaixen_*.py`) in one folder.
2. At the start of `run_simulation.py`: delete `__pycache__/zaixen_*.pyc`.
3. Remove the modules from `sys.modules` and import them again (do not rely on `importlib.reload`).
4. Call `importlib.invalidate_caches()` so new files are seen.
5. Test: print something version-specific (like a new function argument) and check it shows up.

TECHNICAL WAY

Isaac's Python trusted cached bytecode without comparing it to the source, so edited modules silently did not load. We proved it: a class printed the old constructor arguments, while a fresh `compile()` of the file printed the new ones.
Setting `sys.dont_write_bytecode = True` before importing stops new stale caches from being created.
This bug hid several of our fixes for hours, so it is the first thing to suspect when 'the change did nothing'.

3.5 Backups and the session layer

IN SIMPLE WORDS

Before you do anything risky, copy the scene file. And when you only want to try something, do it on a scratch page that is never saved.

STEP BY STEP

1. Copy `zaixen_scene.usd` to a new name (`..._backup_v1.usd`).
2. Do the risky work.
3. Verify. If it is good, save. If not, reopen the backup.
4. For quick experiments use the session layer (see technical box).

TECHNICAL WAY

Session-layer experiment: `stage.SetEditTarget(Usd.EditTarget(stage.GetSessionLayer()))`, make the edit, look, then restore the previous edit target and clear the session spec. Nothing reaches the saved file.
We used this to prove that `Camera_Chase` cannot be deleted without touching the real scene.

3.6 How the scripts run inside Isaac Sim (and why Play alone is not enough)

IN SIMPLE WORDS

The scene file (`.usd`) stores THINGS: roads, cars, lights, cameras, the ego. It cannot store behaviour, because behaviour is code. Think of a board game: the `.usd` is the board and the pieces, and the Python scripts are the players who move them.
So if you only open the file and press Play, physics runs, but nobody moves the cars or changes the lights, and the ego just sits there. You must first run `run_simulation.py` once. That is the step that 'wakes up' the players. After that, Play/Stop work as you expect.

STEP BY STEP

1. Start Isaac Sim (with the Python server on if you also want to send scripts from outside).
2. Open the scene: `File > Open` and choose `zaixen_scene.usd`.
3. Open the Script Editor: `Window > Script Editor`.
4. In the Script Editor: `File > Open` and choose `scripts/run_simulation.py` (or paste its contents).
5. Press **Run** (Ctrl+Enter). Wait for the message 'Simulation armed' and for the **NPC Traffic** panel to appear (bottom right, next to Property).
6. Press **Play** on the left toolbar. The lights change, the cars drive and the ego decides.
7. Press **Stop** to reset everything to the same start. You do NOT need to run the script again after Stop.
8. You DO need to run it again every time you open the scene file again (or restart Isaac Sim).

TECHNICAL WAY

What is saved in the `.usd` and what is not: saved = geometry, materials, cameras, the 30-car pool, settings such as `zaixen:npcCount`. Not saved = anything that is a running callback, timer or subscription.
What `run_simulation.py` does, in order: (1) finds the `scripts` folder next to the scene file and puts it on `sys.path`; (2) asks any previously loaded copy of our modules to `disarm()` (stop their callbacks); (3) deletes cached `.pyc` files and removes the modules from `sys.modules` so they are loaded fresh; (4) imports `zaixen_runtime`, `zaixen_traffic`, `zaixen_ego`, `zaixen_log`; (5) `zaixen_runtime.arm()` subscribes to the physics step, the contact reports and the timeline Stop event; (6) `zaixen_traffic.arm()` registers the lights and NPCs and builds the NPC Traffic panel; (7) `zaixen_ego.arm()` takes over the ego's joint drives and switches off the asset's ROS drive graph; (8) `zaixen_log.arm()` registers the recorder (still off); (9) `zaixen_runtime.reset()` puts everything at t = 0.
After that, everything is driven by the physics step callback: 60 times per simulated second the runtime calls each module's step function in a fixed order (traffic, then ego, then log). That is why nothing moves while stopped.
Could it be automatic? Yes, but not without extra machinery, for example launching Isaac Sim with `--exec path/run_simulation.py`, or packaging the modules as a small Kit extension that arms itself when a stage opens. We kept the manual step because it is simple, visible and easy to debug.

3.7 How the small NPC Traffic window is built (omni.ui)

IN SIMPLE WORDS

The little panel (a label, a slider and a check box) is made with `omni.ui`, Isaac Sim's own tool for windows. You describe the window in code, like stacking building blocks: a window contains a vertical stack, the stack contains a label, a slider and a row with a check box. When you move the slider, a function you wrote is called.
We also had a bug: the text overlapped. Each time the arm script ran, it created a NEW window with the same name while the old ones were still alive, so several panels were drawn on top of each other. The fix was to create the window only once, keep it, and rebuild its contents each time.

STEP BY STEP

1. Make a window: `ui.Window('NPC Traffic', width=360, height=140)`.
2. Open its content area: `with win.frame:`.
3. Add a vertical stack: `with ui.VStack(spacing=8, height=0):`.
4. Add a label with a fixed height: `ui.Label('10 NPC cars driving (max 30)', height=22)`.
5. Make a number holder (a 'model') and a slider that edits it: `model = ui.SimpleIntModel(10)`, `ui.IntSlider(model, min=0, max=30, height=24)`.
6. Add a row with a check box and its text: `with ui.HStack(height=24, spacing=8): cb = ui.CheckBox(width=24); ui.Label('Log to CSV ...')`.
7. Tell it what to do on change: `model.add_value_changed_fn(function)` for the slider, `cb.model.add_value_changed_fn(function)` for the check box.
8. Keep the window object somewhere that survives, and reuse it next time (see technical box).

TECHNICAL WAY

Callbacks: the slider callback calls `set_npc_count(m.as_int)`, which switches cars on or off, refreshes the label text and stores the number in the stage (`zaixen:npcCount`). The check box calls `zaixen_log.enable()` or `disable()`.
Layout: every row has an explicit `height`, and the labels are separate from the slider. Rows given no height, or a window that is docked and resized, are what make text collide.
The overlap bug, exactly: `ui.Window` was created on every arm. Hiding (`visible = False`) does not delete a window, and Isaac cannot delete a native window whose Python object was lost when our module was reloaded (`ui.Workspace.get_windows()` only returns handles, which cannot be destroyed). Old copies stayed, were drawn on top, and their old sliders even wrote a wrong NPC count into the stage.
The fix: keep ONE window object in `builtins` (an attribute that survives our module reloads); on each arm reuse it: `win.frame.clear()` then rebuild inside `with win.frame:`. Stale copies from older versions are hidden through their handles (`handle.visible = False`). Verified: arming twice leaves exactly one visible panel.



Figure 4b - The NPC Traffic panel after the fix: one label, one slider, one check box, nothing overlapping.

3.8 Small everyday tricks

IN SIMPLE WORDS

These are the little things you will do again and again. Keep this page nearby.

STEP BY STEP

1. Pick what you want to do from the table below.
2. Copy the line into a script or the Script Editor.
3. Change the path or the number.
4. Run it and read the output.

TECHNICAL WAY

All of these use `omni.usd`, `pxr` and `omni.kit`. They run inside Isaac Sim (Script Editor or the python server).

I want to...	Code
Get the open stage	`stage = omni.usd.get_context().get_stage()`
Read where a prim is	`UsdGeom.XformCache().GetLocalToWorldTransform(prim).ExtractTranslation()`
Find where a prim was defined (can I delete it?)	`prim.GetPrimStack()` and look at each layer and `specifier` (Def or Over)
Hide or show a prim	`UsdGeom.Imageable(prim).MakeInvisible()` / `.MakeVisible()`
Change a setting on a prim	`prim.GetAttribute('name').Set(value)`, then read it back to check
Change a material colour	`UsdShade.Shader(shader_prim).GetInput('diffuseColor').Set(Gf.Vec3f(r, g, b))`
Look through a different camera	`get_active_viewport().camera_path = '/World/.../Camera_ChassisView'`

I want to...	Code
Take a screenshot of the 3D view	`capture_viewport_screenshot_async(path)` from `isaacsim.test.utils.image_capture`
Take a screenshot of the whole app window	`capture_app_screenshot_async(path)` (this shows panels such as NPC Traffic)
Wait a few frames inside a script	`for _ in range(10): await omni.kit.app.get_app().next_update_async()`
Press Play / Stop from code	`omni.timeline.get_timeline_interface().play()` / `.stop()` (wait a few frames after play before checking `is_playing()`)
Try something without saving it	Set the edit target to the session layer: `stage.SetEditTarget(Usd.EditTarget(stage.GetSessionLayer()))`
Change the NPC count	`zaixen_traffic.set_npc_count(20)`
Check background jobs	`python isaacsim_send.py --introspect tasks`

4. Building the world

Roads, corners, paint, lights and scenery. This is where your scene gets its shape.

4.1 Coordinates, units and the one height rule

IN SIMPLE WORDS

Pick a simple grid before you build anything. Ours: units are metres, X is east, Y is north, Z is up, the middle of the junction is (0, 0). Then pick ONE number for the road surface height and make every road obey it. If roads are at different heights, cars bump and sensors lie.

STEP BY STEP

1. Decide the axes (X east, Y north, Z up) and the units (metres).
2. Put the junction centre at the origin.
3. Choose one road-top height (ours is 0.02 m) and store it as a setting on the scene: `zaixen:roadSurfaceZ`.
4. Make every road piece end exactly at that height.
5. Add a check that reads the top of every road and compares it to the setting.

TECHNICAL WAY

Geometry helpers: `UsdGeom.Cube` with `size=1` and a scale op is the easiest 'box'. A road piece 30 m x 2 m x 0.02 m is a cube scaled (30, 2, 0.02) moved up by 0.01 so its top sits at 0.02.
Custom attributes: `prim.CreateAttribute('zaixen:roadSurfaceZ', Sdf.ValueTypeNames.Double, custom=True).Set(0.02)`. Other scripts read it, so there is a single source of truth.
Right-hand traffic: the lane on the right of the travel direction. Northbound lane x = +0.5, southbound x = -0.5; eastbound y = -0.5, westbound y = +0.5.

4.2 Ground and roads

IN SIMPLE WORDS

The ground is one big flat green slab. Roads are long thin dark slabs sitting on top. Two roads crossing make the junction.

STEP BY STEP

1. Make the ground: a cube 40 x 40 m, 0.1 m thick, top at height 0.
2. Make Road_X: 30 m long, 2 m wide, top at 0.02. Make Road_Y the same but rotated by swapping length and width.
3. Give each a dark grey material.
4. Give the ground a green material.
5. Add colliders (topic 5.1) so things can rest on them.

TECHNICAL WAY

Two lanes per road, 1 m each, so a road is 2 m wide. Arm length 15 m each way.
The ground is a `Cube` at z = -0.05 with scale (40, 40, 0.1), so it spans +-20 m and its top is exactly z = 0.

4.3 Sidewalks with rounded corners (curb returns)

IN SIMPLE WORDS

At the place where a straight road meets the ring road, a square sidewalk corner looks broken. Real streets round that corner. We rounded every one with a small circular arc (0.8 m radius) and filled the gap behind it with asphalt so there is no hole.

STEP BY STEP

1. Draw the sidewalk as one continuous strip that follows the road edge, then curves around the corner, then follows the ring's inner edge.
2. For the curve, find the circle (radius 0.8 m) that touches both the straight road edge and the ring edge.
3. Sample that circle in 1 degree steps to get points.
4. Build the strip as a mesh: a top face, a bottom face and side faces.
5. Fill the small leftover corner between road and curb with a 'fan' of triangles at road height.
6. Check from above and from a low angle that no gap or notch remains.

TECHNICAL WAY

Fillet geometry for an arm of half-width w = 1 and a ring inner radius R = 14 with fillet radius r = 0.8: centre x = w + r = 1.8; centre distance from origin = R - r = 13.2; centre y = sqrt(13.2^2 - 1.8^2) = 13.077. The tangent point on the ring sits at atan2(13.077, 1.8) = 82.16 degrees.

The mesh is built with `UsdGeom.Mesh` (points, face vertex counts, face vertex indices), then `CollisionAPI` + `MeshCollisionAPI` with `approximation='none'` (exact static triangle mesh).

Careful with winding order (which way triangles face) and with degenerate triangles: a triangle whose apex lies on its own base has zero area and breaks physics cooking.

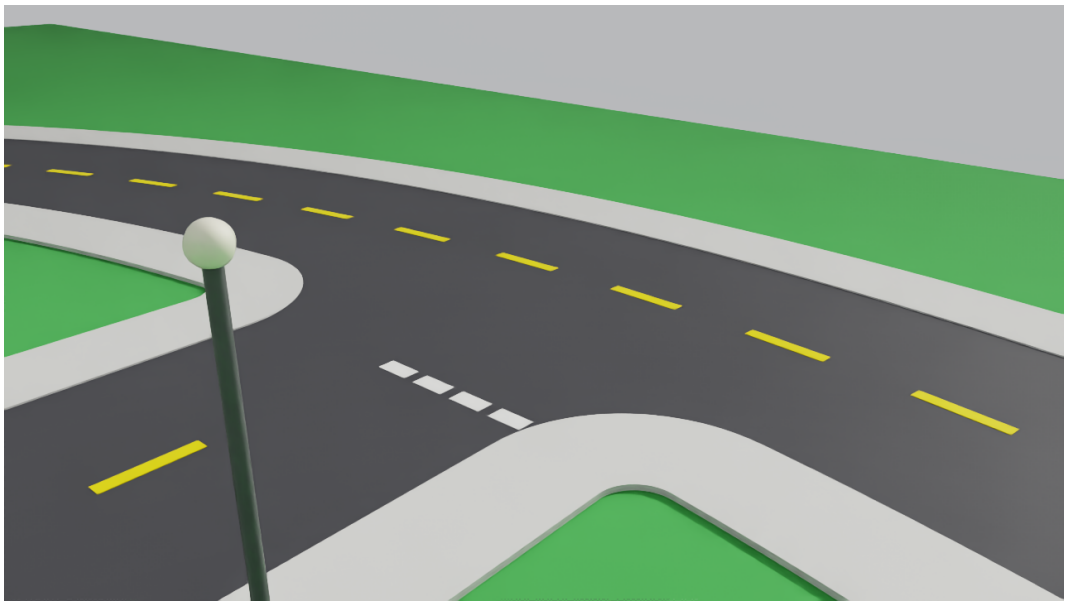


Figure 3 - A ring mouth after the fix: rounded curbs, continuous yellow ring line, dashed white give-way line.

4.4 The ring road

IN SIMPLE WORDS

The ring is a big circle around the plus. It has two lanes: the inside lane goes clockwise, the outside lane goes counter-clockwise, like a roundabout. The four arms of the plus end on it.

STEP BY STEP

1. Choose the ring's centre-line radius (15 m) so it passes through the four arm ends.
2. Make the road a flat ring (annulus) from radius 14 to 16.
3. Make the outer sidewalk from 16 to 16.6.
4. Paint a dashed yellow centre line at radius 15 as separate small boxes placed around the circle.
5. Paint a dashed white give-way line across the exit lane of every arm, just before the ring.

TECHNICAL WAY

Annulus mesh: for each of 360 angles add four vertices (inner/outer, top/bottom) and connect neighbours into quads (top, bottom, inner wall, outer wall).
Lane centres: inside lane r = 14.5 (clockwise), outside lane r = 15.5 (counter-clockwise). Right-hand traffic means clockwise on the inside.
Dash i sits at angle 2*pi*i/n, rotated by (angle + 90 degrees) so its long side follows the circle.

4.5 Road paint

IN SIMPLE WORDS

Paint is just very thin white or yellow boxes lying on the road, a hair above it so they do not flicker.

STEP BY STEP

1. Make a yellow and a white material.
2. Centre dashes: a row of small boxes down the middle of each arm, stopping before the give-way line.
3. Zebra crossing: a row of white bars across the road at each junction entrance.
4. Stop line: one white bar across each approaching lane, just behind the crossing.
5. Put all paint at height 0.021 (road top 0.02 + 1 mm) so it never sinks into the road.

TECHNICAL WAY

Paint boxes use scale z = 0.005 and translate z = 0.021. Same top-of-road rule as 4.1, so anything that overlaps in the same plane (z-fighting) is avoided.
The order along an approach is: open road, stop line (front bumper stops here), zebra crossing, junction. Cars stop with their front bumper at 1.75 m from the centre.

4.6 Materials and lights

IN SIMPLE WORDS

A material is a colour (and how shiny). Lights make the scene bright: one soft light from everywhere (dome) and one 'sun'.

STEP BY STEP

1. Make a material per colour you need (road, sidewalk, grass, paint, car colours, tree green...).
2. Bind each material to the objects that use it.
3. Add one dome light for soft even light and one distant light (the sun) with a slight angle so you get shadows.
4. Look at the result; adjust brightness.

TECHNICAL WAY

`UsdPreviewSurface` shader inputs: `diffuseColor` (Color3f), `roughness` (float), `emissiveColor` (glow). `UsdShade.MaterialBindingAPI(prim).Bind(material)`.
Physics uses a separate binding: `Bind(physics_material, UsdShade.Tokens.weakerThanDescendants, 'physics')`, so the visual colour is untouched.
Our lights: `UsdLux.DomeLight` intensity 400 and `UsdLux.DistantLight` intensity 800, angle 1 degree, rotated (-55, 20, 0).

4.7 Traffic lights that change colour

IN SIMPLE WORDS

Each traffic light has three lamps: red, amber and green. To 'turn one on' we make its material glow; to turn it off we make it almost black. A timer decides which lamp glows.

STEP BY STEP

1. Build a pole and a head box, plus three small spheres (red, amber, green).
2. Give each sphere its own material with a glow setting (`emissiveColor`).
3. Write a function that maps time to a phase: green 8 s, yellow 2 s, red 10 s (a 20 s cycle).
4. The other direction uses the same pattern shifted by half a cycle.
5. Every step, if the phase changed, set the glow of the three materials of that direction.
6. Store the current phase in one shared variable, `TRAFFIC_STATE`, so cars and the ego can read it.

TECHNICAL WAY

Glow values: on = (18, 0, 0) red, (18, 10, 0) amber, (0, 18, 0) green; off = (0.03, 0.03, 0.03). Only the changed axis is written, so the cost is tiny.
The phase function is pure: `signal_phase(t) = GREEN if t%20 < 8, YELLOW if < 10, else RED`; `X = phase(t + 10)`. Because it depends only on simulation time it is fully repeatable.
Nothing else is needed: the lamp materials are part of the scene, and the timer is a callback (chapter 6).

4.8 Scenery that costs almost nothing (PointInstancer)

IN SIMPLE WORDS

If you place 1,500 blades of grass as 1,500 separate objects, Isaac Sim gets slow. Instead you make ONE blade and tell Isaac 'draw it at these 1,500 spots'. That is a PointInstancer. It is how we added about 2,000 props (trees, bushes, hedges, fences, benches, bins, flowers, grass) with almost no slowdown.

STEP BY STEP

1. Build one small model of each thing (a tree = trunk + ball).
2. Put those models inside the instancer as 'prototypes'.
3. Make a list of positions (x, y), a rotation and a size for each copy, and which prototype each one uses.
4. Fill the instancer with those lists.
5. Never give scenery colliders: it must not disturb physics.
6. Choose positions by random sampling, and reject any spot that is on a road, sidewalk, or too close to something else.

TECHNICAL WAY

Author with `UsdGeom.PointInstancer.Define`, add prototype prims as children (`Protos/...`) and targets to `prototypes`, then set `protoIndices`, `positions`, `orientations` (`Quath`) and `scales`.
Random placement uses a fixed seed (`random.Random(2026)`) so the scene is reproducible. Rules: `lawn_ok(x, y)` keeps 0.3 m off sidewalks and 1.3 m off the curb-return circles; `free(x, y, r)` keeps circles apart.
Result: 8 instancers, 1,992 instances; the validator confirms none is on a road and none has a collider.



Figure 4 - Scenery: fence, bench, bin, trees, bushes, flowers and grass tufts, all instanced.

Remember Build in this order: geometry, then colours, then physics, then behaviour. If something looks wrong, go back one layer, not forward.

5. Physics: making things solid and repeatable

Colliders, kinds of bodies, the fixed time step, and how to detect a crash.

5.1 Colliders and friction

IN SIMPLE WORDS

A shape you can SEE is not automatically something you can BUMP INTO. To make a road something a wheel can roll on, you must add an invisible solid shape (a collider) to it. Friction tells physics how grippy each surface is.

STEP BY STEP

1. For every road, sidewalk and the ground: add a collider.
2. Make three physics materials: asphalt (very grippy), pavement, grass.
3. Bind each physics material to the right surfaces.
4. Do NOT add colliders to scenery you never touch.
5. Check with a script that every road piece has a collider and a physics material.

TECHNICAL WAY

`UsdPhysics.CollisionAPI.Apply(prim)`. Static colliders have no `RigidBodyAPI`. Exact-mesh statics use `MeshCollisionAPI` with `approximation='none'`.
Physics materials: `UsdPhysics.MaterialAPI` (static 1.0, dynamic 0.9, restitution 0) plus `PhysxMaterialAPI` with friction combine mode 'max' and restitution combine mode 'min'. 'Max' friction means the grippier of tyre and road wins; 'min' restitution stops the wheel asset's 0.7 bounciness from making the car bounce.
Bound with the 'physics' purpose so visual colours are unaffected.

5.2 Three kinds of moving things

IN SIMPLE WORDS

Static things never move (roads). Dynamic things are pushed around by forces (a robot's wheels). Kinematic things are moved by YOUR code and cannot be pushed, but they do push others. Our NPC cars are kinematic: our code drives them along paths, and if the ego hits one, the ego stops, not the car.

STEP BY STEP

1. Static: only a collider.
2. Dynamic: a rigid body and a collider (mass comes from the shape).
3. Kinematic: a rigid body with `kinematicEnabled = True`, and a collider.
4. Move a kinematic body by changing its transform every physics step.
5. Turn a car 'off' by disabling its rigid body and collider and hiding it.

TECHNICAL WAY

Each NPC root is an Xform with `RigidBodyAPI` (kinematic) and `PhysxContactReportAPI`; its child `Model` carries the 0.6 scale and the `Body` cube has the collider. Putting the scale on a child avoids a scaled rigid-body root.
Static colliders that are moved every frame work, but a kinematic body gives correct contact and velocity, which is what makes the collision with the ego physically right.

5.3 A fixed time step: why runs repeat

IN SIMPLE WORDS

If your cars move by 'real seconds passed', a slow computer moves them differently from a fast one and no two runs match. Instead, make everything advance by the same tiny amount on every physics tick: 1/60 of a second. Then two runs are identical.

STEP BY STEP

1. Set the physics scene to 60 steps per second.
2. Do NOT use the clock (`time.time()`) anywhere in behaviour code.
3. Run your logic inside the physics-step callback, using the step's `dt`.
4. Use seeded random numbers, so 'random' starts are the same each time.
5. On Stop, reset every module to the same start state.
6. Test: run twice and compare positions.

TECHNICAL WAY

`physxScene:timeStepsPerSecond = 60` and `/app/player/useFixedTimeStepping = True`. We subscribe with `omni.physx.get_physx_interface().subscribe_physics_step_events(callback)`; `dt` is exactly 1/60.
Stop is caught with the timeline event stream (`TimelineEventType.STOP`) and calls each module's reset hook. Rendering can run slower than physics without changing results.
Our proof: two 30-second Plays gave ego positions equal to 0.000 mm and identical NPC states, lights and decisions.

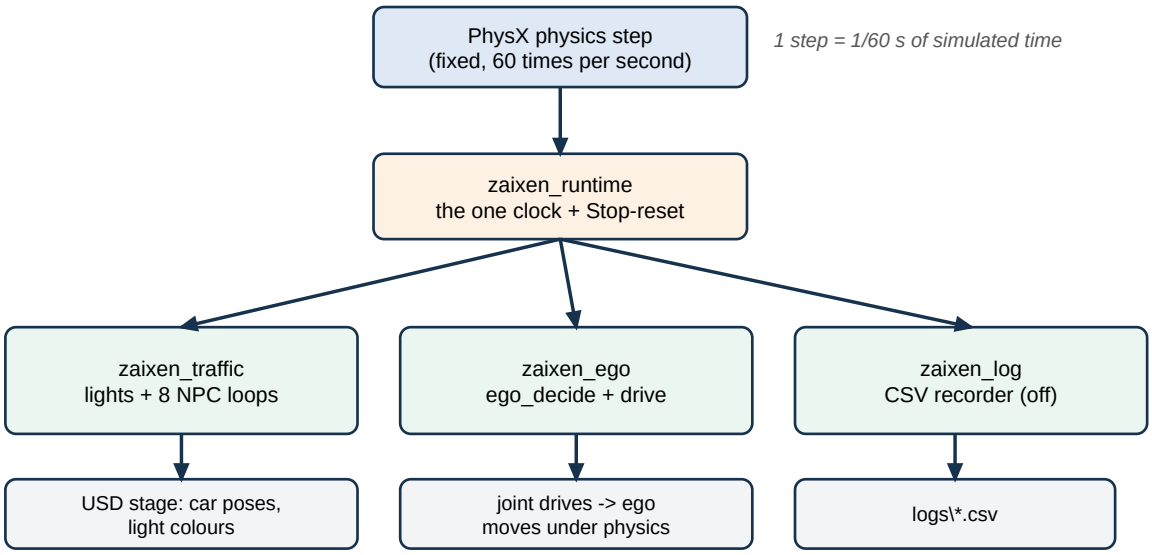


Figure 5 - One clock (the physics step) drives every behaviour. Nothing reads wall-clock time.

5.4 Detecting a crash (contact reports)

IN SIMPLE WORDS

Physics knows when two solid things touch. You can ask it to tell you. We listen for 'the ego touched an NPC car', set a flag, count the crash, and make that car stand still for a second.

STEP BY STEP

1. Add a contact-report setting to the things you care about (the NPC cars).
2. Subscribe to the physics contact events.
3. When a pair of touching objects arrives, look at their names: is one the ego and one an NPC?
4. If yes, set the crash flag and start a 'stand still' timer on that NPC.
5. Count a new crash only if there was no contact for a full second (contact flickers while two things press together).

TECHNICAL WAY

`PhysxSchema.PhysxContactReportAPI.Apply(prim).CreateThresholdAttr(0.0)`; `omni.physx.get_physx_simulation_interface().subscribe_contact_report_events(cb)`. Each header has `actor0/actor1` integers; `PhysicsSchemaTools.intToSdfPath()` turns them into paths, and `ContactEventType.CONTACT_LOST` is skipped.

Our crash test: a deliberately 'blind' ego drove into a parked car. Contact at t = 4.77 s, flag 1, NPC hold 0.98 s, ego stopped with no bounce. The first version counted one crash five times because of flicker; the 1-second episode rule fixed it.

6. NPC traffic: cars that behave

How up to 30 computer-driven cars follow lanes, obey lights, keep a gap and never crash.

6.1 A car's route is a closed loop of lines and arcs

IN SIMPLE WORDS

Every NPC follows a path drawn in advance, like a train on invisible rails. The path is made of straight pieces and circular arcs, joined smoothly, and it comes back to where it began, so the car can drive forever without being teleported.

STEP BY STEP

1. Decide the lanes: which lines and circles a car may drive on.
2. Decide the turns: every turn between an arm and the ring is one circular arc that touches both lanes smoothly.
3. Write down four loops: right-turn loops use the inside ring lane, left-turn loops use the outside lane.
4. Turn those four loops by 90 degrees to get four more for the other arms.
5. Sample each loop into points 4 cm apart. A car is then just 'a distance s along its loop'.

TECHNICAL WAY

Right turns: an arc of radius 1.3 m centred on the curb-return circle, tangent to the arm lane and the inside ring lane. Left turns: radius 2.0 m into the outside lane. The maths: arc centre (x_c, y_c) with distance to the origin equal to (lane radius - arc radius), so the arc touches the ring lane from inside.
Loops: L1/L2 clockwise (right turns), L3/L4 counter-clockwise (left turns), then L5-L8 = the same rotated by (x, y) -> (-y, x), driving the E-W arms under the X signal.
Position and heading are looked up by arclength: `route.xy(s)` interpolates the resampled points; `route.heading(s)` the tangent.

6.2 Speed: accelerate, brake, and slow for corners

IN SIMPLE WORDS

A car does not jump to a new speed. It speeds up gently, brakes smoothly, and slows down BEFORE a tight corner.

STEP BY STEP

1. Pick a cruise speed (about 1 m/s, each car a little different: 0.85 to 1.1 times).
2. Work out a safe speed for every point of the loop from how sharp the curve is.
3. Look ahead so the car starts slowing before the corner, not in it.
4. Every step: find the biggest safe speed given whatever is ahead, then move toward it with limited acceleration (1 m/s^2) and braking (up to 4.5 m/s^2).

TECHNICAL WAY

Curve limit: v = sqrt(a_lat / kappa) with a_lat = 0.55 m/s^2 and curvature kappa from the change of heading over 10 samples; smoothed backwards with v_i = min(v_i, sqrt(v_(i+1)^2 + 2 a ds)).
Stop-at-distance rule used everywhere: the allowed speed is sqrt(2 * a_comfort * distance_free), with a_comfort = 2 m/s^2. It draws a smooth braking curve that ends exactly at the stopping point.
Position update: s += 0.5 (v_old + v_new) dt.

6.3 Obeying the traffic light

IN SIMPLE WORDS

Each car knows how far its front bumper is from the stop line. On red it stops at the line. On yellow it stops if it can do so comfortably; if it is too close it drives on.

STEP BY STEP

1. Find the stop line's position along the car's own loop.
2. Each step, compute the distance from the front bumper to the line.
3. If the light is not green and the line is still ahead: compute the braking distance for the current speed.
4. If the car can stop before the line, aim to stop at it. If not (on yellow only), carry on.
5. A car that already passed the line ignores the light.

TECHNICAL WAY

Braking need: v^2 / (2 * 4.5) on red, v^2 / (2 * 2.0) on yellow. Stop target: line distance minus 0.03 m. Each route has an `axis` (Y or X), so it reads the correct signal.
A car held by its light is marked `red = True`; it must not claim a conflict zone (see 6.5).

6.4 Keeping a gap behind the car in front

IN SIMPLE WORDS

Each car watches the one ahead and stays about 0.3 m behind its bumper.

STEP BY STEP

1. For cars on the same loop: the gap is the difference in their positions along the loop. That is exact.
2. For cars on different loops that share a lane: a car is 'ahead' if it is in front, in the same direction, and within 0.38 m sideways.
3. Stop so that the centres stay 0.8 m apart.
4. The result is a smooth queue at a red light.

TECHNICAL WAY

Same loop: `gap = (o.s - c.s) mod L`; constraint `dfree = gap - 0.8`. Different loops: geometric test in the follower's frame (longitudinal 0 to 3 m, lateral below 0.38 m, heading within 45 degrees).
`dfree` is the distance the car is still allowed to travel; every rule (light, gap, zone, ego) just lowers it, and the car takes the smallest.

6.5 Crossings and merges: conflict zones

IN SIMPLE WORDS

Where two loops cross or join, only one car may be in that spot at a time. We work out these spots in advance from the geometry. The first car to arrive owns the spot; the other waits just before it and goes when it is free.

STEP BY STEP

1. Compare every pair of loops: find stretches where they come within 0.72 m of each other.
2. Ignore stretches where they run on top of each other (that is just following) or peel apart.
3. What remains are crossings and merges: store them as 'zones' with an entry and exit on each loop.
4. Rule 1: a car already inside a zone keeps it; cars before it must wait.
5. Rule 2: if two cars are both about to enter, the one that reaches it sooner goes, and that decision is remembered so it cannot flip.
6. Rule 3: only the head of each queue competes, and a car that cannot actually move (red light, stuck behind someone) claims nothing.

TECHNICAL WAY

Zone search uses a spatial hash of the other route's points; a run of near points that starts on top of the other path is a divergence (ignored), one that ends on top of it is a merge (zone ends at the coincidence). A claim only counts if `hard >= distance + 0.2`, where `hard` is the free distance after light, following and 'inside' constraints. This removed wait cycles such as: car A waits for B, B is stuck behind C, C waits for A. Soak testing found each missing rule: a 3-car cycle at a mouth, claims from stuck cars, and an ego-versus-NPC mutual wait. See chapter 9.

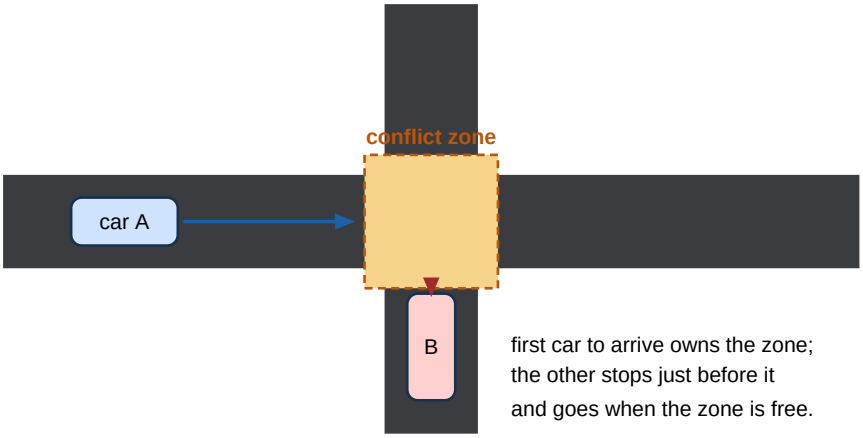


Figure 6 - A conflict zone: two paths cross, one car at a time.

6.6 Yielding to the ego

IN SIMPLE WORDS

The ego always has right of way. Each NPC predicts where the ego will be in the next second and a half, and if its own path comes close, it stops short.

STEP BY STEP

1. Take the ego's position, heading, speed and turn rate.
2. Predict its positions at 0.3, 0.6 ... 1.5 s, following a circular arc if it is turning.
3. Look along the NPC's own path for the first place that comes within about 0.72 m (up to 0.97 m if the ego is turning tightly) of any predicted position.
4. Stop before that place.
5. Exceptions: a car already inside the junction keeps going to clear it; a car whose path merely goes past an ego that is BEHIND it in its own lane ignores it.

TECHNICAL WAY

Constant-turn-rate prediction: x(t) = x + (v/w)(sin(psi + w t) - sin psi), y(t) = y - (v/w)(cos(psi + w t) - cos psi). The margin grows with turn rate because the body corners swing out to about 0.77 m in a tight U-turn. Both exceptions came from real failures: a stopped car frozen in the middle of the junction, and an NPC and the ego each waiting for the other.

6.7 The slider and the pool of cars

IN SIMPLE WORDS

We build 30 cars once and hide the ones you do not need. The slider (or one command) decides how many are 'switched on'. Turning cars on or off never rebuilds anything.

STEP BY STEP

1. Build a pool of 30 cars, all hidden and disabled.
2. To add a car: pick a free spot on the emptiest loop (with clear space around it), place it and switch it on.
3. To remove a car: switch off the highest-numbered active car.
4. Remember the number in the scene so it comes back next time.
5. Show a small window with a slider and a label.

TECHNICAL WAY

`zaixen_traffic.set_npc_count(n)`. Switching a car on toggles `visibility`, `physics:rigidBodyEnabled` and the collider's `collisionEnabled`. The number is stored as `/World/Junction.zaixen:npcCount`.
UI: `omni.ui.Window('NPC Traffic')` with an `IntSlider` on an `SimpleIntModel`, and a check box for the logger.

7. The autonomous ego

A robot car that decides for itself and drives through its real wheels and steering.

7.1 What the vehicle is made of

IN SIMPLE WORDS

The ego is the NVIDIA 'Leatherback': a little four-wheel car with suspension. Its front wheels can turn, and all four wheels have a small motor. To drive it, you do not move it. You tell the motors how fast to spin and the steering how far to turn, and physics does the rest.

STEP BY STEP

1. Find the ego's joints (`Joints` folder in the asset).
2. Two front 'knuckle' joints steer: give them a target angle.
3. Four wheel joints drive: give them a target speed.
4. Write those numbers every physics step.
5. Switch off the asset's ROS drive graph while you drive it yourself, or it will overwrite your numbers with zeros.

TECHNICAL WAY

Steering joints `Knuckle__Upright__Front_Left/Right`: angular position drive (stiffness 1000). Wheel joints `Wheel__Upright__Rear_*`, `Wheel__Knuckle__Front_*`: angular velocity drive (damping 1e8, infinite force). Units are degrees and degrees/second. Positive steering is a left turn; all four wheels positive drives forward (both verified by experiment).
Vehicle numbers from the asset's own Ackermann node: wheelbase 0.32 m, track 0.24 m, wheel radius 0.052 m, max wheel rotation 45 degrees.
Because the wheel drives have infinite force, driving into something solid can make the physics unstable. Our route keeps the ego on the road and a failsafe stops the timeline if its position ever becomes invalid.

7.2 Steering like a real car (Ackermann)

IN SIMPLE WORDS

When a car turns, the inside wheel must turn a bit more than the outside wheel, and the outside wheels must spin faster. Otherwise the tyres fight each other. We compute both from one number: how far the 'steering wheel' is turned.

STEP BY STEP

1. Pick the steering angle you want for the middle of the car (a positive number turns left).
2. Work out the turn radius: R = wheelbase / tan(angle).
3. Left front wheel angle = atan(wheelbase / (R - track/2)). Right front wheel angle = atan(wheelbase / (R + track/2)).
4. Each wheel's speed is proportional to its distance from the turn centre.
5. Send those six numbers to the joint drives.

TECHNICAL WAY

Use `atan`, NOT `atan2`. For right turns R is negative, and `atan2(L, negative)` returns about 174 degrees instead of about -6, which points the wheel backwards. We hit exactly this bug: the ego reversed and drove off the road. A unit test now checks mirror symmetry (turning right by s equals turning left by s with sides swapped).
Wheel speeds: yaw rate w = v / R; rear wheel speed = w * (R -/+ track/2); front wheel speed = w * sqrt((R -/+ track/2)^2 + wheelbase^2); then divide by wheel radius and convert to degrees/s.
Steering is limited to 0.62 rad so the inner wheel stays under the asset's 45 degrees.

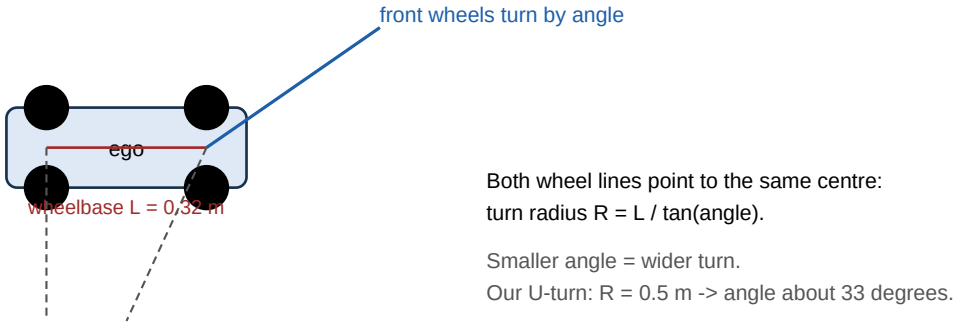


Figure 7 - The 'bicycle model': one steering angle decides the turn radius.

7.3 The route and how the ego follows it

IN SIMPLE WORDS

The ego follows a route made of two straight lanes and two half-circles. To stay on it, it constantly aims at a point a short distance ahead on the route and steers toward that point.

STEP BY STEP

1. Draw the route: east along y = -0.5 to x = 11, a half circle (radius 0.5), west along y = +0.5 to x = -11, another half circle, and repeat.
2. Every step find where you are on the route (search only FORWARD from your last position).
3. Pick the look-ahead point about half a metre ahead (0.45 m in turns).
4. Compute the angle between where you point and where that point is.
5. Turn the front wheels in proportion to that angle.

TECHNICAL WAY

Pure pursuit: steer = atan2(2 L sin(alpha), Ld) where alpha is the heading error to the look-ahead point and Ld the look-ahead distance; the reference point is the rear axle (chassis origin minus 0.16 m).
The U-turn: a rear-axle semicircle of radius 0.5 m needs a steering angle of atan(0.32 / 0.5) = 32.6 degrees, well inside the 45 limit, and its outer body corner sweeps to 0.77 m so it fits the 2 m road.
A forward-only search matters: with a two-way search the ego started next to the END of the loop (equally near) and drove the wrong way.

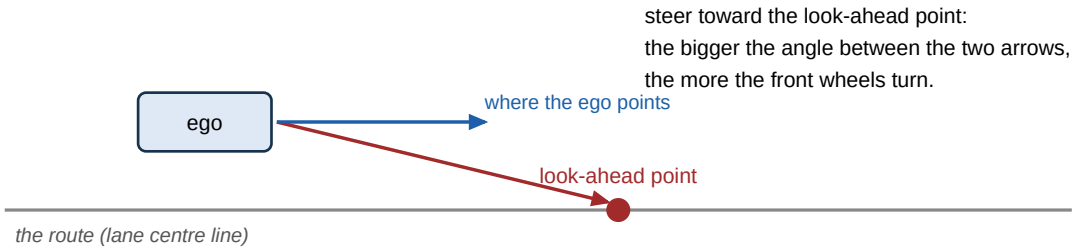


Figure 8 - Pure pursuit: steer toward a point ahead on the route.

7.4 The decision: one clear function

IN SIMPLE WORDS

All the 'thinking' is in ONE function. You give it three facts (what colour the light is, how far the nearest car ahead is, where the ego is) and it tells you whether to stop or go and how to steer. Keeping it in one place means you can lift it out later and put it into another program unchanged.

STEP BY STEP

1. Read the light for the ego's direction (`TRAFFIC_STATE['X']`).
2. Measure the distance to the nearest car ahead.
3. Read the pose (position, heading, speed).
4. Call `ego_decide(light, obstacle_dist, pose, plan)`.
5. Take its answer: stop or go, target speed, steering angle.
6. Turn that into wheel commands (7.2) and write them to the joints.

TECHNICAL WAY

Signature: `cmd, plan = ego_decide(light, obstacle_dist, pose, plan)`, `cmd = Command(go, speed, steer, reason, leg)`; `plan` holds only the progress along the route. No Isaac imports, so it moves into a ROS node unchanged.
Inside, the target speed is the minimum of: cruise (0.9 m/s), turn speed (0.5 m/s) with a smooth slow-down before a turn, the red/yellow stop profile, and the car-ahead stop profile. Reason strings: `go`, `red`, `yellow`, `obstacle`.
Red/yellow rule: stop for the line at 1.75 m unless already too close (yellow only). Car ahead: keep a 0.25 m gap. Inside a U-turn the ego is committed; only a car practically touching stops it.
Outside the function (vehicle glue): after any contact in the last second the drive is cut, and the speed and steering are rate-limited (0.6 and 2.5 m/s^2, 2.5 rad/s) so physics sees smooth demands.

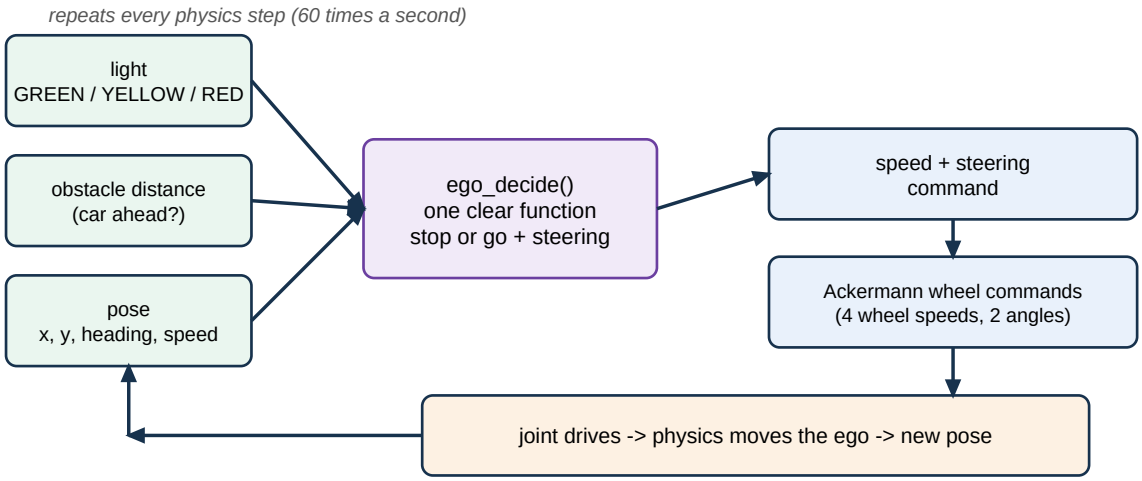


Figure 9 - The ego's loop: inputs -> ego_decide -> wheel commands -> physics -> new pose.

7.5 A stand-in for a sensor

IN SIMPLE WORDS

Later the ego will have cameras or lidar to see cars. For now we cheat honestly: we read the true positions of the cars and check whether any is on the ego's route ahead. The result is a single number (distance to the nearest car ahead), the same number a real sensor would give.

STEP BY STEP

1. Walk along the route ahead in 10 cm steps, up to 3 m.
2. At each point ask: is any car within 0.5 m?
3. The first yes gives the distance.
4. Before a U-turn, look at a wider band (0.95 m) around the arc, because the turn sweeps more area than a lane.
5. If a car is in that wide area, report the distance to the START of the turn so the ego waits before it.

TECHNICAL WAY

`ego_perceive(npcs, pose, plan, look=3.0)` returns metres or None. Replace it with real perception later; the decision function does not change.
The 'wait before the turn' rule was added after long soaks showed the ego starting a U-turn with a car already parked in the swept area.

7.6 Safety nets

IN SIMPLE WORDS

Robots fail in surprising ways. We added small safety nets that only act when something has gone wrong.

STEP BY STEP

1. Invalid pose (not a number, or more than 60 m away): zero the drives and stop the timeline.
2. No valid pose for 2 seconds: same.
3. Touched a car in the last second: cut the drive.
4. On Stop: zero every drive target and reset.
5. When you stop using the in-sim driver, call `zaixen_ego.disarm()`: it zeroes the drives and turns the ROS graph back on.

TECHNICAL WAY

The runtime refuses to read a pose that is not finite or is beyond 100 m (a stale physics pose at Stop looked like 7e12 m and once fed the controller garbage). It also does not read the pose at reset, because physics has not restored the start pose yet.
`disarm()` always re-enables the ROS graph: 'restoring what it was' failed after an interrupted session that had left it disabled.

8. Cameras

How to look at the scene, and how we added a chase camera locked to the ego.

8.1 A camera that is locked to the ego

IN SIMPLE WORDS

If you make a camera a CHILD of the ego's body, it moves wherever the ego moves, like a camera bolted to the car. You only choose where on the car it sits and where it points.

STEP BY STEP

1. Pick the parent: the chassis body.
2. Make a camera under it.
3. Choose the lens: field of view (we used 70 degrees wide).
4. Choose the position: 1.3 m behind, 0.92 m up.
5. Point it forward, tilted only 6 degrees down, so far things stay in view.
6. Check with a screenshot at the start and at the stop line.
7. Switch the viewport to it.

TECHNICAL WAY

Path: `/World/Ego/leatherback/Rigid_Bodies/Chassis/Camera_ChassisView`. The chassis frame is in centimetres (world scale 0.01), so the pose is (-130, 0, 90) in local units. The camera looks along its own -Z with +Y up, so the transform matrix rows are: X = (0, -1, 0), Y = (sin 6, 0, cos 6), Z = (-cos 6, 0, sin 6).
Field of view from the lens: HFOV = 2 atan(aperture / (2 focal)); with aperture 20.955 mm and focal 14.96 mm it is 70 degrees. The old chase camera (60 degrees, 15.8 degrees down) could not see the 1.5 m signal heads closer than about 7 m; the new one sees 21 degrees above the horizon, so they stay visible right up to the stop line.
Switch view: `get_active_viewport().camera_path = '/World/.../Camera_ChassisView'`.



Figure 10 - Through the new camera at the start: the road, the junction and the signals ahead are all visible.

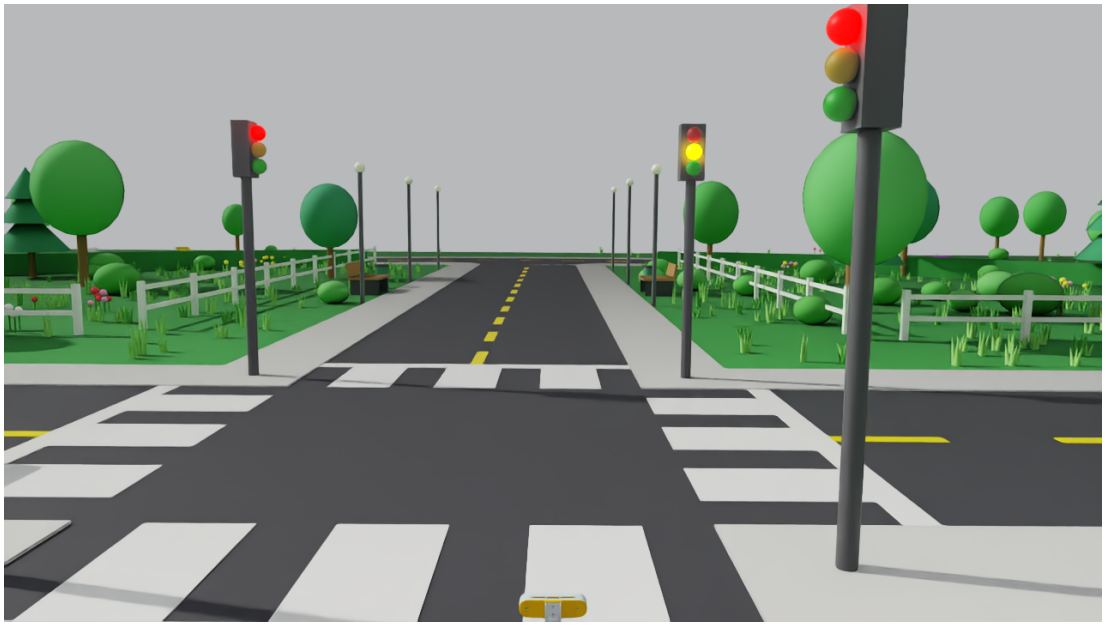


Figure 11 - Waiting at the stop line during a yellow: the signal heads are large and readable.

8.2 Why we could not delete Camera_Chase

IN SIMPLE WORDS

The original chase camera comes from the ego's own model file. Things that come from another file cannot be deleted from your scene, only hidden. So we left it alone and added our own.

STEP BY STEP

1. Ask: where is this object defined? (`prim.GetPrimStack()`).
2. If a `Def` is in another file (a reference), you cannot remove it locally.
3. You may deactivate it (`SetActive(False)`), which hides it.
4. Safest: leave it and add a new one.

TECHNICAL WAY

`stage.RemovePrim(path)` returned False and the prim stayed. The root layer had only an `Over` (the pose you set); the `Def` is in `leatherback.usd`. We tested this in the session layer so the real scene was never touched.

9. Proving it works

Three kinds of tests, and the bugs they caught. This is what turns 'it seems to work' into 'it works'.

9.1 Test the brain without the simulator

IN SIMPLE WORDS

The decision logic is just maths. So we run it in plain Python, thousands of times faster than Isaac Sim, with a pretend car. If the logic is wrong, we find out in seconds, not hours.

STEP BY STEP

1. Keep the logic in files that do not need Isaac Sim (pure Python).
2. Write a tiny pretend world: the car is a simple bicycle model, the cars are the real traffic code.
3. Run it for a long simulated time (30 minutes) with different random starts.
4. Automatically count: collisions, red-light runs, times the car left the road, laps, longest time stuck.
5. Fix the first failure, then run everything again.

TECHNICAL WAY

`test_ring_traffic.py`: NPCs only (with an optional scripted ego). `test_ego_loop.py`: the real `ego_decide` and `ego_perceive` on a bicycle model with speed and steering rate limits at 60 Hz. Collisions use a true oriented-rectangle overlap test (separating axis), stricter than the planner's own circles.
Cost: about 40 seconds of computer time per 30 simulated minutes.

9.2 Soak testing: long runs with many random starts

IN SIMPLE WORDS

A bug that happens once in an hour will not show up in a 5-minute test. So we let the simulation run for hours, on many different random starts, and look only at the totals.

STEP BY STEP

1. Pick several traffic levels (10, 20, 30 cars).
2. For each, run 8 different random seeds for 30 simulated minutes.
3. Print one line per run with the counts.
4. Investigate the FIRST failure: re-run the same seed and print what every car was doing just before.
5. Fix the cause, then re-run the whole soak (not just the failing case).

TECHNICAL WAY

Final results: NPC soaks (4-loop and 8-loop) 32 runs x 30 min each; ego + NPC soak 24 runs x 30 min. Zero collisions, zero red-light runs, no ego stall longer than a red light (11.7 s).
Soak seeds are the reproducible handle on a rare bug: `dbg.py 20 6` means 20 cars, seed 6.

Bugs the tests caught (a real list)

What was seen	Real cause	Fix
Two cars overlapped at a crossing	Priority flipped after a car had already slowed down	Right of way is decided once and remembered
A car stopped inside the other's path	Only the next 1.3 m of the winner's path was considered	Model the crossing car through its whole crossing (3 m)

What was seen	Real cause	Fix
Three cars stuck for 15 minutes	A waited for B, B was stuck behind C, C waited for A	Only the queue head competes for a zone
Cars stuck at the ring mouths	A stopped car still 'owned' a zone it could never reach	A claim counts only if the car can really get through
NPC drove into the ego in a U-turn	NPC predicted the ego in a straight line	Predict along the ego's turn rate; bigger margin in turns
Ego started a turn into a parked car	Corridor only 0.5 m wide, but a turn sweeps more	Wider corridor before the turn; hold before the arc
Ego and NPC each waited for the other	NPC yielded to an ego that was BEHIND it	Ignore an ego behind you in your own lane
Ego drove backwards and off the road	`atan2` gave 174 degrees for a right-turn wheel	Use `atan`; unit test for mirror symmetry
Ego locked onto the wrong part of the route	Search looked backwards to the end of the loop	Forward-only search
Fixes 'did nothing' in Isaac	Stale cached .pyc files	Delete caches, import fresh
One crash counted five times	Contact flickers while pressed together	New crash only after a second of no contact

9.3 The in-app validator

IN SIMPLE WORDS

A script that checks the real scene, with the real physics, and prints PASS or FAIL for 29 things. Run it after every big change and before saving.

STEP BY STEP

1. Open the scene and run `run_simulation.py`.
2. Run `validate_scene.py` (it plays for about 24 simulated seconds, then stops).
3. Read `logs/validation_report.txt`.
4. If any line says FAIL, fix that first.

TECHNICAL WAY

Checks include: rounded corners present, no leftover old sidewalks, every road top at the same height, colliders and physics materials, 60 Hz step, 30 kinematic cars with contact reports, 27 ego colliders, NPC footprint 0.480 x 0.300 m against the ego's 0.476 x 0.305 m, scenery never on a road, runtime armed, ROS graph off, logger off, and live checks (no NPC overlap, signals cycle, ego drives, no crash, Stop resets). Last result, on the scene re-opened from disk: **29 of 29 passed**.

9.4 Checking determinism

IN SIMPLE WORDS

Run the same thing twice and see if the numbers match. If not, something is using real time or luck.

STEP BY STEP

1. Play for 30 seconds, write down the ego position at 5, 10, 15, 20, 25 and 30 s.
2. Stop (this resets everything).
3. Play again and write the same numbers.
4. Compare.

TECHNICAL WAY

Result: maximum ego position difference 0.000 mm, NPC state difference 0, identical lights and decisions, even when the logger was on during the second run.

10. Logging your runs

A file of numbers you can later turn into charts and metrics.

10.1 The CSV recorder

IN SIMPLE WORDS

A recorder writes one line per moment: what time it is, where the ego is, where every car is, what colour the lights are, and whether there was a crash. It is off until you switch it on, so it never slows you down by surprise.

STEP BY STEP

1. Run `run_simulation.py`.
2. Switch logging on: `zaixen_log.enable()` (or tick 'Log to CSV').
3. Press Play and let it run.
4. Press Stop, then `zaixen_log.disable()`.
5. Open the file in `logs/` with Excel or Python (pandas).

TECHNICAL WAY

Path: `D:/Robotics/Projects/Zaixen_simulation/logs/sim_log_<timestamp>.csv`. `enable(every=6)` records 10 rows a second instead of 60. 137 columns: run, step, sim_time, ego pose and speed, the ego's decision, the light states, collision flag and total, NPC count, then x, y, yaw and speed for each of 30 NPC slots.
It runs on the physics step, buffers 120 rows and flushes to disk. The `run` column goes up on every Stop so one file can hold several runs.

Ideas for what to measure later

- Time to complete a lap; time spent waiting at red; number of stops.
- Closest distance to any car (safety margin).
- Number of collisions per hour.
- How much the ego slows down as the number of NPC cars grows (plot speed against NPC count).

11. Saving, backups and your daily routine

How to keep your work safe and how to come back to it tomorrow.

11.1 Saving safely

IN SIMPLE WORDS

Save only when everything is calm: the simulation is stopped, and the ego is back at its start. Keep a backup copy. Then re-open the saved file to prove it works.

STEP BY STEP

1. Stop the timeline.
2. Copy the scene file to a backup name.
3. Switch off the runtime helpers (`disarm`) so their temporary settings are not left in the scene.
4. Save (`omni.usd.get_context().save_stage()`).
5. Re-open the saved file, run `run_simulation.py`, and run the validator.
6. Re-arm and continue.

TECHNICAL WAY

Our saved scene contains the geometry, materials, scenery, the 30-car pool with physics settings, the new camera and the settings attributes (`zaixen:roadSurfaceZ`, `zaixen:npcCount`...). It does NOT contain running behaviour: callbacks, timers and subscriptions are runtime only, which is why `run_simulation.py` exists.
Isaac Sim may autosave; do not assume a backup copy is 'clean' unless you made it yourself.

11.2 Your daily routine

IN SIMPLE WORDS

Every time you come back to the project: open the scene, run one script (`run_simulation.py`), then press Play. Opening the scene and pressing Play on its own is NOT enough: the scene file holds the objects, but the lights, cars and ego are Python behaviour that only starts after that script runs (see topic 3.6). You do not need to run it again after Stop.

STEP BY STEP

1. Start Isaac Sim (with its Python server on) and open `zaixen_scene.usd`.
2. In the Script Editor run `scripts/run_simulation.py`.
3. Press Play. Watch. Press Stop to reset.

TECHNICAL WAY

`run_simulation.py` disarms anything old, deletes stale bytecode, imports the modules fresh, arms runtime, traffic, ego and log (in that order), and resets everything to t = 0.

The re-arm commands

What	Command
Everything	run `scripts/run_simulation.py`
Clock / physics step	`zaixen_runtime.arm()`
Lights and NPC cars	`zaixen_traffic.arm()`
Ego driver	`zaixen_ego.arm()` (undo: `zaixen_ego.disarm()`)

What	Command
CSV recorder	`zaixen_log.arm()` then `zaixen_log.enable()`
NPC count	`zaixen_traffic.set_npc_count(14)` or the slider

12. When something goes wrong

Symptom, likely cause, what to do. Start at the top and work down.

Symptom	Most likely cause	What to do
I changed the code and nothing changed	Stale cached Python files	Run `run_simulation.py` (it clears caches). Check with a version-specific print.
Script works alone but the screen shows nothing	The scene was not saved, or the wrong stage is open	Print `omni.usd.get_context().get_stage_url()` and check.
The connection dropped and I lost the output	Job longer than about a minute	Use `--fire-and-forget` and write results to a file.
Nothing moves when I press Play	Modules not armed, or Play not pressed	Run `run_simulation.py`, then Play. Nothing moves while stopped, by design.
Cars move but the ego does not	ROS graph is overwriting the drives, or ego not armed	`zaixen_ego.arm()`; check the ROS drive graph is disabled while driving.
The ego reverses or veers off at the start	Wrong steering sign or wrong route start	Run the wheel-command unit test; check the route index starts at 0.
The ego's position is a huge number	Physics exploded (hit something solid with infinite-force wheels)	The failsafe stops the timeline; find why it left the road.
Cars overlap or stop forever	A rule is missing in the priority logic	Run the soak, print the first failure with the debug trace, fix that one cause.
A light never changes colour	Signal materials not found	Check the paths `/World/Looks/Signal_X_Red/Shader` etc. exist.
Everything got slower	Too many separate prims, or scenery has colliders	Use PointInstancer; remove colliders from scenery.
A prim will not delete	It comes from a referenced file	Leave it, or deactivate it (`SetActive(False)`), or add a new one.
Top view looks rotated	A top-down camera has no fixed 'up'	Not a bug. Work out orientation from a known object (the ego).
'Set attribute' does nothing at runtime	Wrong prim path, or attribute is on the wrong layer	Read the value back right after setting it.

13. How to explain this to other people

Three sizes of explanation, and short answers to the questions you are most likely to be asked.

The 30-second version (anyone)

We built a small town in a robot simulator: roads, a roundabout-style ring road, traffic lights and trees. Computer-driven cars drive around it obeying the lights, and one self-driving robot car has to drive through the same traffic without hitting anyone. The simulation is exact, so we can repeat any run and measure how safely the robot drives.

The 2-minute version (a friend who knows a little tech)

The scene is described in USD, a file format for 3D worlds, and built with Python scripts: roads and curbs from simple shapes and small bits of geometry, thousands of trees and grass blades stamped cheaply with instancing, and traffic lights whose bulbs glow on a timer. The other cars are 'on rails': each follows a pre-computed loop of lines and arcs, speeds up and brakes smoothly, keeps a gap, and where two loops cross, a first-come-first-served rule decides who goes. The robot car is different: every physics tick it looks at the light colour and the distance to any car ahead and decides whether to stop or go and how to steer, then drives through its real wheel motors and steering, so collisions are real physics. Everything runs on a fixed 60-times-a-second clock, so every run is identical.

The technical version (an engineer)

- **Scene:** USD stage with a single `roadSurfaceZ` source of truth; curb returns as tangent fillet arcs (r = 0.8) triangulated as static exact-mesh colliders; scenery via `UsdGeom.PointInstancer` (about 2,000 instances, no colliders).
- **Determinism:** PhysX at 60 Hz, all behaviour on the `subscribe_physics_step_events` callback with fixed dt, seeded RNG, reset on timeline STOP. Verified bit-identical ego trajectories.
- **NPCs:** 8 closed loops (4 turned 90 degrees), 4 cm resampled polylines, curvature-limited speed profile, a single 'free distance' constraint per car from light / gap / zone / ego, precomputed pairwise conflict zones with head-of-queue arbitration, remembered priority, and claims only from cars that can actually clear the zone. Kinematic rigid bodies with contact reports.
- **Ego:** the Leatherback articulation driven through joint drive targets (Ackermann steering angles as position targets; wheel speeds as velocity targets); pure-pursuit path tracking; one pure function `ego_decide(light, obstacle_dist, pose, plan)`; ground-truth `ego_perceive` as a sensor stand-in; failsafes; ROS drive graph disabled while active.
- **Verification:** off-line closed-loop soaks (about 44 simulated hours in total), in-app validator (29 checks), determinism check, deliberate collision test.

Questions people ask and short answers

Question	Simple answer	Technical answer
How did you make the roads?	Flat dark boxes on a green box, with thin white and yellow boxes for paint.	Scaled `UsdGeom.Cube` prims; road top height from one shared attribute; paint at z = 0.021.
How are the curved corners made?	A small circle is fitted into each corner and the sidewalk follows it.	Fillet circle tangent to the arm edge and the ring edge; sampled every 1 degree into a triangle mesh with an exact static collider.
How do you get so many trees so cheaply?	One tree is drawn once and copied to many spots.	`PointInstancer` with prototypes and per-instance position/orientation/scale arrays.
How do the traffic lights work?	Each bulb's material glows when it should be on; a timer picks the colour.	Emissive colour of three materials per axis, set from a pure function of simulation time.

Question	Simple answer	Technical answer
How do the other cars know where to go?	They follow a fixed loop like a train, but they brake for lights and other cars.	Arclength-parameterised closed polylines; every rule reduces a free distance; speed follows sqrt(2 a d).
How do they avoid crashing?	At each crossing, the first car to arrive goes; the other waits just before it.	Precomputed conflict zones; priority decided once and remembered; only the queue head competes.
Is the robot car just replaying a path?	No. It checks the light and the cars every moment and chooses.	`ego_decide` is evaluated each physics step from live state; the path only defines steering targets.
How does it steer?	Like a real car: the inner front wheel turns more.	Ackermann geometry from wheelbase and track; pure pursuit on the route; rate-limited commands to joint drives.
How do you know it will not crash?	We ran it for many hours with many random starts and counted crashes: none.	Soaks over seeds and traffic levels with an OBB overlap test; deadlocks and near-misses found and fixed.
Why do the runs repeat exactly?	Everything moves by the same tiny time step, never by the real clock.	Fixed 1/60 s physics step callbacks, seeded RNG, full reset on Stop; measured 0.000 mm difference.
What if it does hit a car?	The car stops, the crash is written down, and the robot stops driving.	Kinematic body + contact report; flag and episode counter; NPC hold 1 s; ego drive cut.
Can it drive with ROS?	The brain is one function that can move into a ROS program; the ROS link itself is the next step.	`ego_decide` has no simulator imports; the asset's ROS Ackermann graph is re-enabled by `zaixen_ego.disarm()`.

Tip If you are asked something you cannot answer, say what you would CHECK, not what you guess. For example: 'I would run the validator and read the soak output.' That is honest and it is the method this project used.

14. Build your own from zero

A roadmap of small milestones. Each one has a 'done when' check so you know you can move on.

#	Milestone	Done when
1	Get a script running in Isaac Sim from outside (python server) and read the stage	You can print the list of prims
2	Decide axes, units and the one road height; make ground and one road	A check script confirms the road top height
3	Add a crossing road, sidewalks, paint, colours and lights	Top view and low view both look right
4	Add colliders and physics materials; drop a box on the road	It lands and stops (does not fall through)
5	Build traffic lights with a timer	The colours change in the right order
6	Add ONE car that follows a path by arclength	It drives a loop smoothly, forever
7	Add speed control (accelerate, brake, slow for corners)	No sudden jumps in speed
8	Make it obey the light and keep a gap; add a second car	Cars queue at red and never overlap
9	Move all behaviour onto the fixed physics step; add reset on Stop	Two Plays give identical numbers
10	Add several loops and conflict zones; write the off-line soak test	A 30-minute soak with 0 crashes
11	Add the robot vehicle, learn its joints, drive it in a straight line, then steer it	Straight line, then a left and a right turn both work
12	Write the pure decision function and route following; test off-line first	Ego passes the off-line soak
13	Connect it to the real drives; add failsafes; test in the app	Ego drives a full loop and stops at red
14	Add collisions, logging and the validator; save; re-open and re-check	Validator passes on the re-opened file

Ten habits that will save you days

1. Change one thing at a time and check it straight away.
2. Numbers first, screenshots second.
3. Keep one source of truth for shared values (height, radius, speeds).
4. Never use the real clock in behaviour; use simulation time.
5. Test the logic off-line before you put it in the simulator.
6. Soak test: many random starts, long runs, count the failures.
7. When it fails, re-run the SAME seed and print what happened just before.
8. Add a safety net for every dangerous thing (bad pose, crash).
9. Back up before risky steps; re-open what you saved.
10. Write down the gotchas (the stale cache, the atan bug) so you never lose that time again.

Exercises to try alone

- Change the light timing (say 12 s green) and see how the queues change.
- Set the NPC count to 30 and look at how the ego's speed changes (log it).
- Add a new bench or tree type to the scenery and re-run the scenery script.
- Add a second camera on the ego's left side and look through it.
- Change the cruise speed of the ego and re-run the off-line soak before trying it live.
- Make a fifth NPC loop that goes straight over the ring without turning (hint: new route pieces, then re-run the zone search).

Appendix A - Cheat sheet

The commands you will use most.

I want to...	Command
Send a script to Isaac Sim	`python isaacsim_send.py --file my_script.py`
Run something long	`python isaacsim_send.py --fire-and-forget --file my_script.py`
See background task status	`python isaacsim_send.py --introspect tasks`
Pass a value into a script	`--arg name=value`
Arm everything after opening the scene	run `scripts/run_simulation.py`
Change NPC count	`import zaixen_traffic; zaixen_traffic.set_npc_count(20)`
Start / stop the CSV log	`import zaixen_log; zaixen_log.enable()` / `zaixen_log.disable()`
Give the ego back to ROS	`import zaixen_ego; zaixen_ego.disarm()`
Off-line NPC soak	`python scripts/test_ring_traffic.py --long`
Off-line ego soak	`python scripts/test_ego_loop.py --long`
Full in-app check	run `scripts/validate_scene.py`, then read `logs/validation_report.txt`
Save the scene	Stop, back up, `zaixen_*.disarm()`, `omni.usd.get_context().save_stage()`
Look through the chase camera	`get_active_viewport().camera_path = '/World/Ego/leatherback/Rigid_Bodies/Chassis/Camera_ChassisView'`

Appendix B - Key numbers

Handy facts about this scene.

Thing	Value
Units, axes	metres; X east, Y north, Z up; junction centre at (0, 0)
Arm length / road width / lane width	15 m / 2 m / 1 m
Road surface height	0.02 m
Ring	centre radius 15 m; inside lane r = 14.5 (clockwise); outside lane r = 15.5 (counter-clockwise)
Curb return radius	0.8 m
NPC turn radii	right turn 1.3 m, left turn 2.0 m
NPC size / ego size	0.480 x 0.300 m / 0.476 x 0.305 m
NPC pool / default count	30 cars / 10
NPC cruise speed	about 1.0 m/s (each car 0.85 to 1.1 times)
Traffic light cycle	20 s: 8 s green, 2 s yellow, 10 s red; other axis shifted 10 s
Stop line (front bumper)	1.75 m from the junction centre
Physics step	1/60 s (60 Hz)
Ego cruise / turning speed	0.9 m/s / 0.5 m/s
Ego U-turn	semicircle radius 0.5 m at x = +11 and x = -11; steering about 33 degrees
Ego vehicle	wheelbase 0.32 m, track 0.24 m, wheel radius 0.052 m
Cameras on the chassis	Camera_Right, Camera_Left, Camera_Chase (from the asset), Camera_ChassisView (new: 70 degrees wide, 6.8 degrees down)
Scenery	8 instancers, 1,992 instances, no colliders
Validator	29 checks, all passing

Appendix C - Where everything is

Files in the project folder `D:/Robotics/Projects/Zaixen_simulation`.

File or folder	What it is
`zaixen_scene.usd`	The final scene (saved)
`zaixen_scene_backup_v1.usd`	The scene before any of this work (clean original)
`zaixen_scene_backup_v2_autosave_1440.usd`	An Isaac autosave from the middle of the work
`REPORT_Zaixen_Simulation.md`	The full report of everything that was done
`docs/Beginner_Guide_Zaixen_Simulation.pdf`	This guide
`docs/images/`	Screenshots used in the guide
`logs/`	CSV logs, the example log and `validation_report.txt`
`scripts/run_simulation.py`	Run this after opening the scene
`scripts/zaixen_runtime.py`	The fixed-step clock
`scripts/zaixen_traffic.py`	Signals and NPC traffic
`scripts/zaixen_ego.py`	The ego: `ego_decide`, perception, wheel commands
`scripts/zaixen_log.py`	CSV recorder
`scripts/validate_scene.py`	29-point check
`scripts/test_ring_traffic.py`, `scripts/test_ego_loop.py`	Off-line tests and soaks
`scripts/part1_...`, `part2_...`, `part4_scenery.py`, `step*.py`	The scripts that build the world, in order

Appendix D - The maths, in one place

For when you need the formulas.

Idea	Formula	Meaning
Turn radius	`R = L / tan(steer)`	L is the wheelbase; bigger steering angle gives a smaller circle
Ackermann front wheels	`left = atan(L/(R - T/2))`, `right = atan(L/(R + T/2))`	T is the track width; positive R is a left turn; use atan
Pure pursuit	`steer = atan2(2 L sin(alpha), Ld)`	alpha = heading error to a point Ld ahead
Stop at a distance	`v = sqrt(2 a d)`	Highest speed from which you can stop within d at comfortable deceleration a
Curve speed	`v = sqrt(a_lat / kappa)`	kappa is curvature (1 / radius)
Field of view	`FOV = 2 atan(aperture / (2 focal))`	Wider aperture or shorter focal length gives a wider view
Fillet centre	`x = w + r`, `y = sqrt((R - r)^2 - x^2)`	Circle of radius r touching an arm of half-width w and the ring of inner radius R
Constant-turn prediction	`x(t) = x + (v/w)(sin(psi + w t) - sin psi)`	Where a turning vehicle will be in t seconds